

Beej's Guide to C — Companion Exercise Sheet

Chapter-by-chapter practice problems

How to use this sheet

Chapter 1 — Foreword

Chapter 2 — Hello, World!

Chapter 3 — Variables and Statements

Chapter 4 — Functions

Chapter 5 — Pointers—Cower In Fear!

Chapter 6 — Arrays

Chapter 7 — Strings

Chapter 8 — Structs

Chapter 9 — File Input/Output

Chapter 10 — typedef: Making New Types

Chapter 11 — Pointers II: Arithmetic

Chapter 12 — Manual Memory Allocation

Chapter 13 — Scope

Chapter 14 — Types II: Way More Types!

Chapter 15 — Types III: Conversions

Chapter 16 — Types IV: Qualifiers and Specifiers

Chapter 17 — Multifile Projects

Chapter 18 — The Outside Environment

Chapter 19 — The C Preprocessor

Chapter 20 — structs II: More Fun with structs

Chapter 21 — Characters and Strings II

Chapter 22 — Enumerated Types: enum

Chapter 23 — Pointers III: Pointers to Pointers and More

Chapter 24 — Bitwise Operations

Chapter 25 — Variadic Functions

Chapter 26 — Locale and Internationalization

Chapter 27 — Unicode, Wide Characters, and All That

Chapter 28 — Exiting a Program

Chapter 29 — Signal Handling

Chapter 30 — Variable-Length Arrays (VLAs)

Chapter 31 — goto

Chapter 32 — Types Part V: Compound Literals and Generic Selections

Chapter 33 — Arrays Part II

Chapter 34 — Long Jumps with setjmp, longjmp

Chapter 35 — Incomplete Types

Chapter 36 — Complex Numbers

Chapter 37 — Fixed Width Integer Types

Chapter 38 — Date and Time Functionality

Chapter 39 — Multithreading

Chapter 40 — Atomics

Chapter 41 — Function Specifiers, Alignment Specifiers/Operators

Milestone Projects

How to use this sheet

This sheet follows **Beej's Guide to C Programming v0.10.5** chapter by chapter. Read a chapter, then immediately do its exercises before moving on. Nothing here requires material from a later chapter.

Difficulty markers: ~ warm-up (10 min) · + standard (20–40 min) · ! stretch (1 hr+)

How to compile. Start with this and nothing else:

```
gcc -o program program.c
```

Then run it with `./program` (or `program.exe` on Windows). That's all you need for the first few chapters.

Extra flags get introduced later, one at a time, at the chapter where they start earning their keep. Each one is in a box like this:

Toolchain — what to add, and why it matters here.

Don't add them early. A wall of flags you don't understand hides more problems than it finds, and some of them aren't available on every compiler — if a flag makes your build fail, drop it and keep going. The code is what you're learning, not the command line.

Reaching ahead. Everything an exercise needs has been covered by the time you reach it, with a handful of unavoidable exceptions — a cast, an `unsigned char`, a `const`. Those are marked in the solutions as *[ahead: Ch N]*. Copy them and move on; the reason arrives in that chapter.

Rules for getting value out of this:

1. Write the code before reading the solution. A failed attempt teaches more than a read solution.
2. If you're stuck for more than 20 minutes, reread the relevant section of the chapter — not the solution.
3. After you check the solution, retype your own version from scratch. Don't copy-paste.
4. Keep every exercise in a git repo. Commit per chapter. You'll want the history later.

Chapter 1 — Foreword

No exercises. Setup checklist instead:

- ☐ `gcc --version` (or `clang --version`) works from your terminal
- ☐ You can compile and run a file without an IDE
- ☐ `mkdir ~/bgc && cd ~/bgc && git init`
- ☐ `gcc -o hello hello.c` on a one-line test file produces a runnable `./hello`

Chapter 2 — Hello, World!

2.1 ~ Hello, build. Write `hello.c` that prints `Hello, World!`. Compile it three ways: with no flags, with `-Wall -Wextra`, and with `-o greet`. Confirm `echo $?` prints 0 after running it.

2.2 ~ Break it on purpose. Take your working `hello.c` and make four separate broken copies: one missing the `#include`, one missing the semicolon, one with `Main` instead of `main`, one with a missing closing brace. Compile each. Write down the *first* error line the compiler prints for each. You are training yourself to read compiler output, which is a skill on its own.

2.3 + Version check. Write a program that prints the value of the `_STDC_VERSION` macro. Then compile it with `-std=c99`, `-std=c11`, and `-std=c17` and note how the number changes. (Hint: the value is a `long`, so use `%ld`.)

Chapter 3 — Variables and Statements

Toolchain — turn on warnings

```
gcc -Wall -Wextra -o program program.c
```

`-Wall` and `-Wextra` make the compiler tell you about code that is legal but almost certainly wrong: a variable you set and never read, a `printf` format that doesn't match its argument, a comparison that's always true. None of these stop compilation, so without the flags you never hear about them. From here on, treat a warning as a bug you haven't found yet.

(`-Wall` does *not* mean “all warnings” despite the name. `-Wextra` adds a second batch. There are dozens more.)

3.1 ~ Temperature table. Print a table converting Celsius to Fahrenheit from -40 to 100 in steps of 10, formatted in two aligned columns. Use a `for` loop and a `printf` field width.

3.2 + FizzBuzz, three ways. Print 1–100 with the usual FizzBuzz rules using an `if/else if` chain. Then rewrite it twice more: once as a `switch` on `i % 15`, and once using only the ternary operator. All three use nothing beyond this chapter. Compare which one you'd actually want to maintain.

3.3 + Increment order. Predict, on paper, what this prints, then run it:

```
int i = 5;
printf("%d %d\n", i++, i);
int j = 5;
printf("%d\n", j++ + ++j);
```

Write a paragraph explaining what happened. One of these two lines is *undefined behaviour* — identify which, and explain why the “obvious” answer isn't guaranteed.

3.4 + Digit sum and reverse. Start with a hardcoded `int n = 91827`; . Print the sum of its digits and the number with its digits reversed (e.g. 1234 → sum 10, reversed 4321). Then change `n` to a negative value and make it handle that sensibly too. Use only `%`, `/`, and a loop — no strings, no input functions.

Chapter 4 — Functions

4.1 ~ Prototype practice. Write `int max3(int a, int b, int c)` and `int min3(int a, int b, int c)`, both *defined below* `main()` with prototypes above it. Then delete the prototypes and observe the compiler error you get.

4.2 + Swap that doesn't. Write `void swap(int a, int b)` that swaps its parameters, and call it from `main`. Confirm it does nothing to the caller's variables. Add a comment explaining exactly why. (You'll fix this in Chapter 5 — leave it broken for now.)

4.3 + Recursion pair. Write `factorial(n)` and `fib(n)` recursively, then write iterative versions of both. Keep `n` small enough to fit in an `int` (factorial stops being safe past 12). Then run `fib(35)` both ways — time it from your shell with `time ./program` — and note the difference.

4.4 ! Number classifier. Write `bool is_prime(int n)`, `int gcd(int a, int b)` (Euclid's algorithm, iterative), and `bool is_perfect(int n)` (equals the sum of its proper divisors). Then write a `main` that prints all perfect numbers under 10000 and all twin prime pairs under 200.

Chapter 5 — Pointers—Cower In Fear!

Toolchain — debug info and the sanitizers

```
gcc -Wall -Wextra -g -fsanitize=address,undefined -o program program.c
```

`-g` keeps variable and line-number information in the binary, so crash reports name the actual line instead of a hex address.

`-fsanitize=address,undefined` is the big one. It makes your program check itself as it runs and report reads past the end of an array, use of freed memory, and undefined behaviour like signed overflow — with a stack trace pointing at the exact line. Without it, those bugs usually don't crash; they quietly corrupt something and crash somewhere else ten minutes later. Pointers are where that starts, which is why it shows up in this chapter.

If this flag fails to compile, you're likely on MinGW or plain Windows GCC, which doesn't support the sanitizers. Drop `-fsanitize=...` and carry on with `-Wall -Wextra -g` — you'll just have to be more careful. If you want them, WSL, Linux, or macOS clang all have them.

Sanitizers slow the program down noticeably. That's fine while learning; you turn them off for release builds.

5.1 ~ Address tour. Declare an `int`, a `char`, and a `double`. Print the address of each with `%p`, and its size with `sizeof`. Then declare a second `int` and print its address too. Note whether the two `int`s ended up adjacent, and whether you think you can rely on that.

5.2 + Fix the swap. Rewrite Exercise 4.2 as `void swap(int *a, int *b)` so it actually works. Then write `void swap3(int *a, int *b, int *c)` that rotates three values left.

5.3 + Multiple return values. Write `void divmod(int num, int den, int *quot, int *rem)` that returns both results through pointers. Have it return nothing when `den == 0` and leave the outputs untouched — then change it to `bool divmod(...)` returning success, and argue in a comment which design is better.

5.4 + NULL discipline. Write `void safe_increment(int *p)` that increments `*p` only if `p` is not `NULL`. Call it with a valid pointer and with `NULL`. Then deliberately remove the `NULL` check and run it under the address sanitizer to see what the crash report looks like.

Chapter 6 — Arrays

6.1 ~ Array stats. Given a hardcoded `int` array, compute and print the min, max, sum, and mean. Get the length with the `sizeof` idiom, not a magic number. Keep every value small enough that the sum fits comfortably in an `int`.

6.2 + Reverse and rotate. Write `void reverse(int *arr, int len)` that reverses in place, then `void rotate(int *arr, int len, int k)` that rotates left by `k` using three reversals. Test with `k` larger than `len` and with `k` negative.

6.3 + Matrix work. Declare a 4x4 `int` matrix. Write functions to print it, transpose it in place, and compute the sum of each row and column. Pass the matrix to functions correctly — note what the parameter type has to be.

6.4 ! Sieve of Eratosthenes. Use a `bool` array to find all primes below 1000. Print them 10 per line, right-aligned in columns. Then modify the sieve to also report the largest prime gap it found.

Chapter 7 — Strings

7.1 ~ Length and copy by hand. Write `size_t my_strlen(char *s)` and `void my_strcpy(char *dst, char *src)` without using any `string.h` function. Use array indexing — `s[i]` — rather than moving the pointer itself; the pointer-walking version is Chapter 11's job. Verify against the real ones.

7.2 + String toolkit. Write `void to_upper(char *s)`, `void reverse_str(char *s)`, and `bool is_palindrome(const char *s)` (ignoring case and non-letters). Use `ctype.h`.

7.3 + Word counter. Given a hardcoded sentence, count the words, the letters, and the vowels. A "word" is a run of non-space characters. Handle multiple consecutive spaces and leading/trailing spaces correctly.

7.4 ! Buffer overflow, deliberately. Declare `char buf[8];` and `strcpy` a 20-character string into it. Compile with the

address sanitizer and read the full error report carefully. Then fix it with a `strlen` bounds check before the copy. Finally, explain in a comment why `strcpy` is *not* the safe fix people think it is. (The solution also shows the `snprintf` fix, which is the one you'll actually use — that function arrives in Chapter 9.)

Chapter 8 — Structs

8.1 ~ Point and rectangle. Define `struct point { double x, y; }` and `struct rect { struct point tl, br; }`. Write functions for the distance between two points, and the area and perimeter of a rect.

8.2 + Struct by value vs pointer. Write `void scale_by_value(struct rect r, double f)` and `void scale_by_ptr(struct rect *r, double f)`. Show that only one of them affects the caller. Print `sizeof(struct rect)` and explain what's being copied.

8.3 + Student records. Define a `struct student` with a name array, an id, and an array of 5 grades. Make an array of 4 students, compute each one's average, and print a formatted table sorted by average descending.

Chapter 9 — File Input/Output

9.1 ~ Read and echo. Write a program that opens a text file by a hardcoded name and prints it to stdout one character at a time with `fgetc`. Check `fopen` for `NULL` and report the error with `perror`.

9.2 + Line numbering. Rewrite 9.1 using `fgets` to read a line at a time, prefixing each line with its number, right-aligned in a width of 4. Handle lines longer than your buffer without corrupting the numbering.

9.3 + Write and read back. Write a program that writes 10 lines of `name score` pairs to a file, then reopens it and reads them back with `fscanf` into an array, printing the highest score. Check the return value of `fscanf` every time.

9.4 ! Binary round-trip. Define a struct with an int, a double, and a fixed char array. Write an array of 5 of them to a binary file with `fwrite`, then read them back with `fread` into a fresh array and verify every field matches. Then open the binary file in a text editor and describe what you see.

Chapter 10 — typedef: Making New Types

10.1 ~ Cleaning up structs. Take your `struct student` from 8.3 and `typedef` it. Rewrite the whole program to use the new name. Do it both ways — separate `typedef` and combined `typedef struct { ... } Student;` — and note which one still lets you write a self-referential struct.

10.2 + Typedef traps. Write `typedef char *string;` then declare `string a, b;` and check with `sizeof` and assignment whether both are pointers. Then do the same with `typedef int arr5[5];` and pass an `arr5` to a function. Write up what surprised you.

Chapter 11 — Pointers II: Arithmetic

11.1 ~ Walk an array with a pointer. Sum an `int` array using only a moving pointer and a comparison against `arr + len` — no index variable anywhere.

11.2 + Pointer subtraction. Write `int index_of(int *arr, int len, int target)` that finds an element by scanning with a pointer, then returns the index by *subtracting pointers*. Return -1 if not found. Print the type of the subtraction result using `%td`.

****11.3 ! Generic swap with void*.** Write `void memswap(void *a, void *b, size_t size)` that swaps any two objects of equal size byte by byte. Test it on ints, doubles, and structs. Then write `void reverse_generic(void *base, size_t n, size_t size)` using it.

Chapter 12 — Manual Memory Allocation

Toolchain — leak checking

The address sanitizer from Chapter 5 already reports leaks when the program exits. Run your program normally and read what it prints at the end — `LeakSanitizer: detected memory leaks` followed by the allocation stack trace tells you exactly which `malloc` was never freed.

If you can't use the sanitizers, Valgrind does the same job on Linux with no special compile flags:

```
gcc -Wall -Wextra -g -o program program.c
valgrind --leak-check=full ./program
```

From this chapter on, “it works” isn't enough. An exercise is done when it produces the right output *and* reports zero leaks.

12.1 ~ malloc basics. Ask the user for `n`, allocate an `int` array of that size with `malloc`, fill it with squares, print it, and free it. Check the return of `malloc`. Run under the sanitizer and confirm zero leaks.

12.2 + calloc vs malloc. Allocate the same array twice, once with `malloc` and once with `calloc`, and print both before writing anything. Explain the difference in output — and why the `malloc` version's output isn't something you can rely on.

12.3 + Growable array. Write a `struct vec { int *data; size_t len, cap; }` with `vec_init`, `vec_push`, `vec_get`, and `vec_free`. Double the capacity with `realloc` when full. Push 1000 elements and confirm no leaks.

12.4 ! Read a whole file into memory. Write `char *read_file(const char *path, size_t *out_len)` that reads a file of arbitrary size into a single heap buffer using `realloc` growth, NUL-terminates it, and returns it. The caller frees it. Handle a missing file by returning `NULL`. Test on an empty file and a 1 MB file.

Chapter 13 — Scope

13.1 ~ Shadow hunt. Write a program with a global `int count`, a function-local `count`, and a block-local `count`, all with different values. Print all three from the innermost block (you'll need to restructure to reach the outer ones). Then compile with `-Wshadow` and read the warnings.

13.2 + Static locals. Write `int next_id(void)` that returns 1, 2, 3, ... on successive calls using a `static` local. Then write `void reset_ids(void)` in the same file that resets it, and explain why `reset_ids` can touch that variable but a function in another file could not.

Chapter 14 — Types II: Way More Types!

Toolchain — picking a language version

```
gcc -std=c17 -Wall -Wextra -g -o program program.c
```

Without `-std`, GCC picks its own default, which varies by version and includes non-standard extensions. Pinning it means your code either compiles everywhere or fails loudly.

`c17` is the safe choice — every GCC and Clang from the last several years accepts it. Use `c99` or `c11` if yours doesn't. The newest standard is C23, spelled `-std=c23` on recent compilers and `-std=c2x` on slightly older ones; if neither is accepted, your compiler is too old for it and you should stay on `c17`. Only a handful of exercises late in this sheet need anything past C11, and they say so.

14.1 ~ Size and limits tour. Print `sizeof` and the `<limits.h>` min/max for `char`, `short`, `int`, `long`, `long long`, and each unsigned variant. Print `<float.h>` values for `float`, `double`, `long double`.

14.2 + Unsigned wraparound. Write a loop `for (unsigned int i = 5; i >= 0; i--)` and predict what happens before

running it. Then explain the fix. Separately, print `(unsigned)-1` and explain the value you get.

14.3 + Overflow demo. Set an `int` to `INT_MAX` and add 1. Set an `unsigned int` to `UINT_MAX` and add 1. One of these is undefined behaviour and one is fully defined — say which is which and why. Compile with `-fsanitize=undefined` and see which one it catches.

Chapter 15 — Types III: Conversions

15.1 ~ Safe string to number. Write `bool parse_int(const char *s, int *out)` using `strtol`. It must reject empty strings, trailing garbage, and values outside `int` range. Use `errno` and `endptr`. Test with `"42"`, `"42abc"`, `" "`, `" 42"`, `"9999999999999999"`.

15.2 + Integer division surprises. Predict then verify the output of: `5/2`, `5.0/2`, `5/2.0`, `(float)5/2`, `(float)(5/2)`, `-7/2`, `-7%2`. Write one sentence each explaining the result.

15.3 + Char conversions. Write functions converting a digit character to its `int` value and back, without assuming ASCII values numerically (use `'0'` arithmetic). Then write a hex-digit parser that handles both cases. Explain why `getchar()` returns `int` and not `char`.

Chapter 16 — Types IV: Qualifiers and Specifiers

16.1 ~ const in three positions. Declare `const char *a`, `char * const b`, and `const char * const c`. For each, try to modify the pointer and the pointed-to data. Record which four of the six operations fail and why.

16.2 + const-correct function. Rewrite `my_strlen` and a `count_char` function to take `const char *`. Then try to write to the parameter inside and read the error. Explain what `const` bought you as the *caller*.

16.3 + static and extern. Make a two-file program where `file1.c` has a `static int` hidden and a non-static `int` shared, and `file2.c` tries to use both via `extern`. Get the linker error for one, success for the other. Keep this — you'll extend it in Chapter 17.

Chapter 17 — Multifile Projects

Toolchain — stop typing the command, write a Makefile

Once a project has more than one `.c` file, retyping the compile line stops being viable. Put this in a file called `Makefile` in your practice directory:

```
CC      = gcc
CFLAGS = -Wall -Wextra -g -fsanitize=address,undefined

%: %.c
$(CC) $(CFLAGS) $< -o $@
```

Then `make ex0301` builds `ex0301.c` into `ex0301`. That indent before `$(CC)` must be a **real tab**, not spaces — it's make's oldest and most annoying rule.

Exercise 17.1 has you write a proper multi-target version that compiles each file separately and links them.

17.1 + Split the vector. Take your `struct vec` from 12.3 and split it into `vec.h`, `vec.c`, and `main.c`. Write proper include guards. Compile with a Makefile that builds `.o` files separately and links them.

17.2 ! Header discipline audit. In your split project, deliberately break each of these one at a time and record the exact error: (a) remove the include guard and include `vec.h` twice; (b) define a function in the header instead of the source and link two `.c` files that both include it; (c) put a global variable definition (not `extern` declaration) in the header. Write down the

difference between a *compiler* error and a *linker* error.

Chapter 18 — The Outside Environment

18.1 ~ argv tour. Print `argc`, every `argv[i]` with its index, and confirm that `argv[argc]` is `NULL`. Run it with zero, one, and five arguments, including one with spaces in quotes.

18.2 + Command-line calculator. Accept `prog NUM OP NUM` (e.g. `prog 7 x 3`) and print the result. Use your `parse_int` from 15.1. Exit with `EXIT_FAILURE` and a message on `stderr` for bad input, division by zero, or wrong argument count. Verify the exit status with `echo $?.`

18.3 + Environment reader. Print the value of `PATH` and `HOME` using `getenv`, handling the case where a variable doesn't exist. Then have your program read a variable named `MYAPP_DEBUG` and print extra output only when it's set.

Chapter 19 — The C Preprocessor

19.1 ~ Macro pitfalls. Define `#define SQUARE(x) x * x` and evaluate `SQUARE(2 + 3)` and `100 / SQUARE(5)`. Explain both wrong answers, then write the correct version with full parenthesization.

19.2 + Debug logging. Write a `DBG(fmt, ...)` variadic macro that prints file, line, and function (`__FILE__`, `__LINE__`, `__func__`) plus your message to `stderr` — but compiles to nothing when `NDEBUG` is defined. Test both builds.

19.3 ! ARRAY_LEN and a swap macro. Write `#define ARRAY_LEN(a)` using `sizeof`. Then write a multi-statement `SWAP(a, b, type)` macro wrapped in `do { ... } while (0)` and explain what breaks if you use plain braces instead. Show a case where `ARRAY_LEN` silently gives the wrong answer.

Chapter 20 — structs II: More Fun with structs

20.1 ~ Designated initializers. Initialize a nested struct and an array of structs using designated initializers only, skipping some fields. Print every field and confirm what the skipped ones contain.

20.2 + Padding investigation. Define `struct a { char c; int i; char d; }` and `struct b { int i; char c, d; }`. Print `sizeof` both and the `offsetof` of every member. Explain the difference, then reorder `struct a` to shrink it.

20.3 + Linked list. Build a singly linked list of `struct node { int val; struct node *next; }` with push-front, push-back, print, find, delete-by-value, and free-all. Have the functions that can change the first node *return* the new head rather than taking a pointer-to-pointer — that technique is Chapter 23, and Exercise 23.2 revisits this. Confirm zero leaks under the sanitizer.

20.4 ! Tagged union. Build a `Value` type: an `enum` tag plus a union of `int`, `double`, and `char *`. Write `value_print` that switches on the tag, and `value_free` that only frees when the tag says string. Make an array of mixed values and print them all.

Chapter 21 — Characters and Strings II

21.1 ~ Escape sequence table. Print a table showing each common escape (`\n`, `\t`, `\\`, `\"`, `\0`, `\a`) with its numeric value, using `%d` on the character constant. Then print a string containing a literal backslash and a literal quote.

21.2 + Hex dump. Write a function that takes a string and prints each byte as two hex digits, 16 per line, with the printable ASCII shown in a column on the right (like `xxd`). Run it on a string containing `\t` and non-ASCII bytes.

Chapter 22 — Enumerated Types: enum

22.1 ~ Days and directions. Define `enum day` and `enum direction`. Write `const char *day_name(enum day d)` using a switch, then rewrite it with a lookup array of strings. Note which version the compiler warns about when you add a new enum member.

22.2 + State machine. Model a traffic light with an `enum state`, a `next_state` function, and a loop that steps through 10 transitions printing each. Add an invalid state and see what your switch does.

Chapter 23 — Pointers III: Pointers to Pointers and More

23.1 ~ Double indirection. Write `void alloc_and_fill(int **out, int n)` that allocates an array and hands it back through the parameter. Call it, use the array, free it. Draw the two levels of indirection on paper first.

23.2 + Array of strings. Build a `char **` array of names on the heap (each name individually `malloc`'d and copied). Write functions to print them, sort them alphabetically, and free them all. Confirm no leaks.

23.3 ! Function pointers. Write a `qsort`-style `void my_sort(void *base, size_t n, size_t size, int (*cmp)(const void *, const void *))` using your `memswap` from 11.3 and a simple insertion sort. Use it to sort ints ascending, ints descending, and strings, by passing three different comparators. Then do the same job with the real `qsort` and compare.

Chapter 24 — Bitwise Operations

24.1 ~ Binary printer. Write `void print_bits(unsigned int v)` that prints all 32 bits, grouped in nybbles. Use it to visualise `<<`, `>>`, `&`, `|`, `^`, and `~` on a few values.

24.2 + Bit manipulation kit. Write `set_bit`, `clear_bit`, `toggle_bit`, `test_bit`, and `count_bits` (population count) for an `unsigned int`. Then write `bool is_power_of_two(unsigned)` in one expression.

24.3 ! Flags and a bitset. Define permission flags (`READ = 1<<0`, etc.) and write functions to add, remove, and test them on a single `unsigned`. Then build a bitset over a `unsigned char` array that can hold 1000 bits, with set/clear/test/count operations that compute the right byte and bit offsets.

Chapter 25 — Variadic Functions

25.1 ~ Sum of many. Write `int sum_all(int count, ...)` using `va_start`, `va_arg`, `va_end`. Then write `double avg_all(int count, ...)` taking doubles. Explain why the count parameter (or a sentinel) is unavoidable.

25.2 + Wrapping printf. Write `void log_msg(const char *level, const char *fmt, ...)` that prefixes a level tag and then forwards to `vfprintf(stderr, fmt, args)`. This is the pattern nearly every real logging library uses.

Chapter 26 — Locale and Internationalization

26.1 ~ Locale effects. Print a formatted number and a decimal fraction before and after `setlocale(LC_ALL, "")`. On a system with locales installed, try `"de_DE.UTF-8"` and observe the decimal separator change.

26.2 + localeconv report. Call `localeconv()` and print the decimal point, thousands separator, currency symbol, and sign positions for the C locale and for your system's default. Explain why hardcoding `.` as the decimal separator is a bug in international software.

Chapter 27 — Unicode, Wide Characters, and All That

27.1 ~ Bytes vs characters. Take a UTF-8 string with accented characters or an emoji. Print `strlen` of it and then count the actual characters by counting bytes that are not continuation bytes (`(b & 0xC0) != 0x80`). Explain the gap.

27.2 + Wide character round trip. Convert a multibyte string to a `wchar_t` array with `mbstowcs` and back with `wcstombs`, printing both and the lengths. Use `setlocale` first and note what happens if you forget it.

Chapter 28 — Exiting a Program

28.1 ~ Exit handlers. Register three functions with `atexit` and confirm the order they run in. Then compare `exit()` versus `_Exit()` versus returning from `main` for whether handlers and stream flushing happen.

28.2 + assert in anger. Write a function with an `assert` on a precondition. Trigger it and read the message. Then rebuild with `-DNDEBUG` and confirm the assert vanishes. Explain why you should never put a side effect inside an `assert`.

Chapter 29 — Signal Handling

29.1 ~ Catching SIGINT. Install a `SIGINT` handler that sets a `volatile sig_atomic_t` flag. Loop until the flag is set, then clean up and exit. Press Ctrl-C to test.

29.2 + What you may not do. Write a handler that calls `printf` and one that only writes to the flag. Read the chapter's list of async-signal-safe functions and write a comment explaining precisely why the `printf` version is unsafe even though it appears to work.

Chapter 30 — Variable-Length Arrays (VLAs)

30.1 ~ Runtime-sized array. Read `n` from the user, declare `int arr[n]`, fill and print it. Print `sizeof(arr)` and note that it's computed at runtime.

30.2 + 2D VLA function. Write `void print_matrix(int rows, int cols, int m[rows][cols])` and call it with two differently-shaped matrices. Note the required parameter order. Then write a comment on why you would often use `malloc` instead for large sizes.

Chapter 31 — goto

31.1 ~ Breaking out of nested loops. Write a search over a 3D array that jumps out of all three loops on a hit using `goto`. Then rewrite it with flag variables and compare readability honestly.

31.2 + Cleanup ladder. Write a function that opens two files and allocates a buffer, where any step can fail, using the `goto fail_stage_n` cleanup pattern so each failure frees exactly what was acquired. This is the single most defensible use of `goto` in C.

Chapter 32 — Types Part V: Compound Literals and Generic Selections

32.1 ~ Compound literals. Pass an unnamed struct directly to a function using a compound literal. Do the same with an unnamed array. Then take the address of a compound literal and explain its lifetime.

32.2 + Type-generic macro. Use `_Generic` to write a `TYPE_NAME(x)` macro returning a string naming the type, and a `my_abs(x)` macro dispatching to `abs`, `labs`, or `fabs` by argument type.

Chapter 33 — Arrays Part II

33.1 ~ static in parameters. Write `int sum(int n, int arr[static n])` and call it correctly, then call it with a shorter array and see whether your compiler warns. Explain what the `static` is promising.

33.2 + Qualified array parameters. Write a function taking `const int arr[const 10]` and try to modify both the elements and the parameter itself. Record which fails.

Chapter 34 — Long Jumps with `setjmp`, `longjmp`

34.1 ~ Basic long jump. Set a jump point in `main`, call a function three levels deep, and `longjmp` back with a value. Print the value returned by `setjmp` on both the first pass and the jump-back.

34.2 ! Fake exceptions. Build TRY/CATCH/THROW macros over `setjmp/longjmp` and use them for a division-by-zero “exception”. Then write a comment listing at least three reasons this is a bad idea in real code (start with what happens to local variables and to allocated memory).

Chapter 35 — Incomplete Types

35.1 ~ Opaque handle. Design a `Counter` module where `counter.h` only declares `typedef struct counter Counter;` plus `counter_new`, `counter_inc`, `counter_get`, `counter_free`, and the full struct lives only in `counter.c`. Confirm `main.c` cannot see the fields, and cannot do `sizeof(Counter)`.

35.2 + Forward declaration. Write two structs that point at each other (`struct a` holds a `struct b *` and vice versa). Get the ordering right. Then try making one hold the other by value and read the error.

Chapter 36 — Complex Numbers

36.1 ~ Complex arithmetic. Add, multiply, and divide two `double complex` values, printing real and imaginary parts with `creal` and `cimag`. Verify one result by hand.

36.2 + Mandelbrot in ASCII. Use `double complex` and `cabs` to render a small Mandelbrot set as text in the terminal, iterating $z = z^2 + c$ with an escape test.

Chapter 37 — Fixed Width Integer Types

37.1 ~ Exact widths. Print `sizeof` and the min/max of `int8_t`, `int16_t`, `int32_t`, `int64_t` and their unsigned versions. Use the correct `PRId32/PRId64` format macros from `<inttypes.h>` rather than guessing at `%d` vs `%ld`.

37.2 + Portable struct. Rewrite the binary-file struct from Exercise 9.4 to use fixed-width types only, and write a comment explaining what problem that solves when the file moves between machines. Bonus: note what it *doesn't* solve (endianness).

Chapter 38 — Date and Time Functionality

38.1 ~ Now, formatted. Get the current time with `time()`, convert with `localtime()`, and print it three ways using

`strftime`: ISO 8601, a long human-readable form, and just the weekday name.

38.2 + Date arithmetic. Write a function that takes a year, month, and day, builds a `struct tm`, normalizes it with `mktime`, and reports the day of the week. Use it to check your Chapter 3 leap-year logic against several February 29ths. Then compute the number of days between two dates with `difftime`.

Chapter 39 — Multithreading

39.1 ~ Two threads. Launch two threads with `thrd_create` that each print 5 messages, then `thrd_join` both. Run it several times and note that the interleaving changes. (Compile with `-pthread`.)

39.2 ! Counter race, then fixed. Have 4 threads each increment a shared `long` 100000 times with no synchronization. Print the total and observe it's wrong. Then fix it with an `mtx_t` mutex and confirm it's right. Time both versions.

Chapter 40 — Atomics

40.1 ~ Atomic counter. Redo 39.2 a third time using `atomic_long` and `atomic_fetch_add` instead of a mutex. Confirm correctness and compare the timing against the mutex version.

40.2 + Flag handoff. Use an `atomic_bool` with release/acquire ordering so one thread publishes data and another reads it only after the flag is set. Write a comment explaining what a relaxed ordering would fail to guarantee here.

Chapter 41 — Function Specifiers, Alignment Specifiers/Operators

41.1 ~ inline and noreturn. Write a small `inline` function in a header and use it from two source files, getting the `extern inline` declaration right. Then mark a fatal-error function `noreturn` and confirm the compiler stops warning about missing returns after calling it.

41.2 + Alignment. Print `alignof` for the basic types and for a struct. Then declare a buffer with `alignas(64)` and verify its address is 64-byte aligned by checking the low bits. Use `memalignment()` if your compiler supports it.

Milestone Projects

Do these *between* chapter blocks. They're where separate skills get welded together.

Milestone A — after Chapter 12. A command-line word frequency counter. Read a text file, split into words, count occurrences, print the top 20 sorted by count. Uses: strings, arrays, structs, file I/O, dynamic allocation. Constraint: no fixed limit on file size or number of distinct words.

Milestone B — after Chapter 24. A generic hash table library (`ht.h/ht.c`) with string keys and `void *` values, chaining for collisions, and automatic resizing at a load factor of 0.75. Then rewrite Milestone A to use it and compare the runtime on a large book from Project Gutenberg. Uses: multifile, pointers to pointers, function pointers, bitwise (for the hash), opaque types.

Milestone C — after Chapter 38. A small CSV-backed contact manager: load records at startup, support add/list/search/delete from a command loop, save on exit. Uses: everything above plus parsing, `strtol/strtod`, `enum` for commands, `time.h` for timestamps, and a `goto` cleanup ladder on the load path.

Milestone D — the graduation project. Pick one and finish it: a `grep` clone with your own pattern matcher, a JSON parser, a stack-based virtual machine with its own bytecode, or the `kilo` text editor built from scratch rather than typed in. Put it on GitHub with a README, a Makefile, and tests.